Future & Promise

son: 5 · Concurrency · Category: Concurrency · Level: Intermediate · Build with W

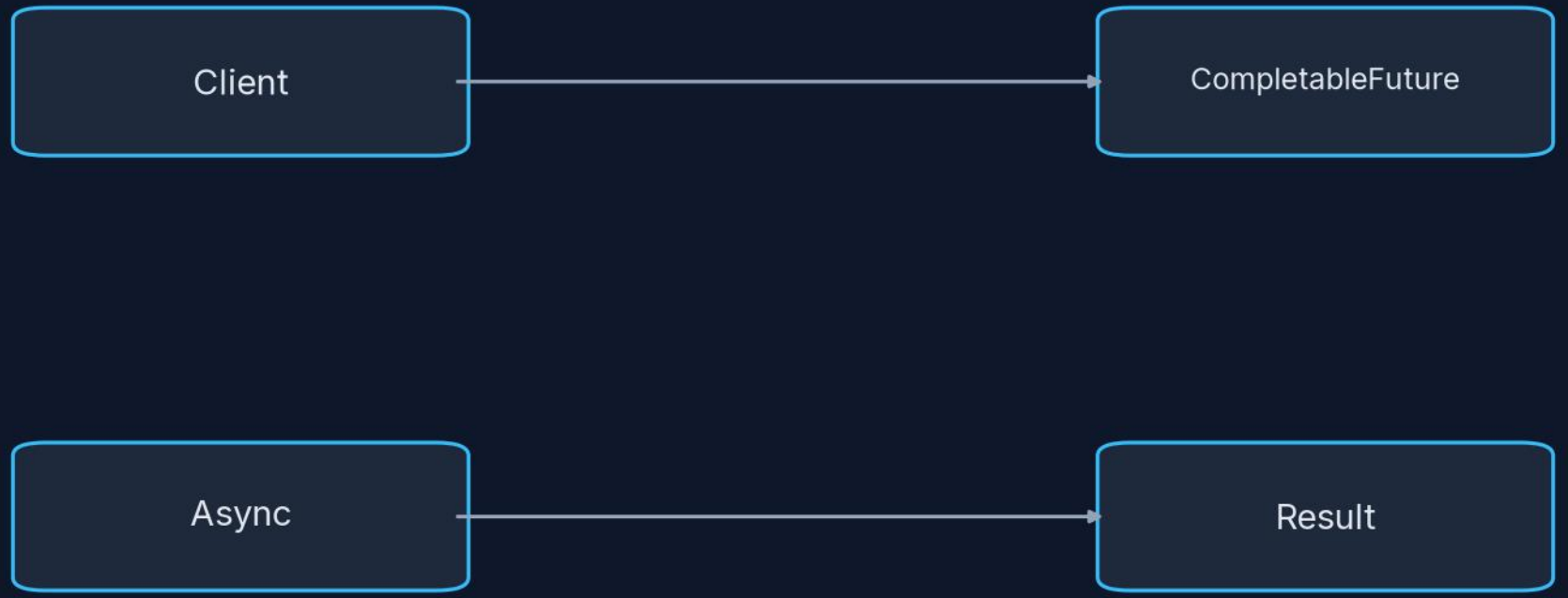
Today's Hook

- Today: Future & Promise as a StreamForge product force — not trivia.
- Runnable Demo on JDK 25 with captured stdout.
- Classic encoding → Java 25 / Zazr modernization → trade-offs.
- CapCut slides + full spoken script included.

The Problem

- Naive local fixes: more ifs, statics, nulls, or threads.
- Illegal states still compile; tests become order-dependent.
- Design reviews lack a shared name for the force.
- Interviews punish memorization without judgment.

Structure



Future & Promise structure

Real-World Scenario

- StreamForge creator platform is the running metaphor.
- Same names appear in naive code, refactor, and Demo output.
- Stay in one story for the full 10–12 minutes.
- Product language first; pattern vocabulary second.

Naive Code

```
// Naive sketch for Future & Promise
public class Naive {
  // duplicated logic / leaked details / unsafe sharing
  Object doIt(Object... args) {
    if (args == null) return null;
    return args[0];
  }
}
```

Why Naive Hurts

- Coupling spreads across controllers and infrastructure.
- Tests need sleeps or static reset hooks.
- New variants require shotgun edits.
- On-call cannot reason about failure modes.

Implementation

```java
// file: src/main/java/com/buildwithwaqar/patterns/concurrency/futurepromise/Promise.java
package com.buildwithwaqar.patterns.concurrency.futurepromise;

import java.util.concurrent.CompletableFuture;
import java.util.concurrent.CompletionStage;
import java.util.concurrent.Future;

/** Promise/future demo using CompletableFuture (Java's standard promise). */
public final class Promise {
    private Promise() {}

    public static CompletionStage<Integer> computeAsync(int x) {
        return CompletableFuture.supplyAsync(() -> x * 2);
    }

    public static Future<Integer> asFuture(int x) {
        return computeAsync(x).toCompletableFuture();
    }
}
```

Demo.main

```
package com.buildwithwaqar.patterns.concurrency.futurepromise;
public final class Demo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Future/Promise walkthrough ===");
    System.out.println("async=" + Promise.computeAsync(21).toCompletableFuture().get());
    System.out.println("future=" + Promise.asFuture(3).get());
  }
}
```

Walkthrough Output

```
=== Future/Promise walkthrough ===
async=42
future=6
```

Step-by-Step Trace

- Each stdout line maps to a refactor stage.
- Re-run Demo locally on JDK 25 to verify.
- Differences usually mean a dirty tree — pull main.
- Pause the video and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction for Future & Promise
// records + sealed + switch
// Option/Either/Validation instead of null
// virtual threads for blocking fan-out
// inject interfaces at the edge
```

Trade-offs

- Clarity vs ceremony — do not spawn five types for two lines.
- Closed sealed worlds vs open SPI plugins.
- Teaching demos omit production idempotency/metrics.
- Prefer simpler language features when they erase the force.

Related Patterns

- Contrast with the neighbor pattern (force difference).
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Force → naive pain → Future & Promise → Demo stdout.
- Modernize with Java 25 / Zazr when it shrinks code.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run Demo.